C++ INDEX

Danishwer

0026MCA25

SQL-INDEX

C++

Danishwer

0026MCA25

S.no: 1

Program Title: Program to check the given number is prime or not

Source code

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{

    int x;
    cout << "Please enter the no. here : ";
    cin >> x;

    if (x <= 1)
    {
        cout << x << " is not prime";
    }
    else
    {
        int limit = sqrt(x);
        bool isPrime = true;
        for (int i = 2; i <= limit; i++)
        {
            if (x % i == 0)
            {
                cout << x << " is not prime";
                isPrime = false;
                break;
            }
        }
        if (isPrime)
        {
            cout << "The number is prime";
        }
    }

    return 0;
}
```

TEST CASES:

Case 1;

Input = 2

```
Please enter the no. here : 2
The number is prime
```

Case 2;

Input = 10

```
Please enter the no. here : 10
10 is not prime
```

Case 3;

Input = 7

```
Please enter the no. here : 7
The number is prime
```

S.no: 2

Program Title: Program to check the given number is Armstrong

Source code

```
#include <iostream>
#include <cmath>
using namespace std;
bool isArmstrong(int);
int digitCount(int);
int main()
{
    for (int i = 0; i < 100000; i++)
    {
        if (isArmstrong(i))
        {
            cout << i << endl;
        }
    }
    return 0;
}
int digitCount(int x)
{
    if (x == 0)
        return 1;
    int dCount = 0;
    while (x > 0)
    {
        x /= 10;
        dCount++;
    }
    return dCount;
}
bool isArmstrong(int x)
{
    int value = x;
    int digits = digitCount(x);

    int sum = 0;

    while (x > 0)
    {
        sum += pow(x % 10, digits);
        x /= 10;
    }
    return value == sum;
}
```

OUTPUT:

```
PS C:\Users\danny\OneDrive\Desktop\MCA\CPP\AssignmentFIle
programs\" ; if ($?) { g++ armstrong.cpp -o armstrong } ;
0
1
2
3
4
5
6
7
8
9
370
371
407
1634
8208
9474
54748
92727
93084
```

S.no: 3

Program Title: Program to find the factorial of the given number

Source code

```
#include <iostream>
using namespace std;

int factorial(int n)
{
    return (n <= 1) ? 1 : n * factorial(n - 1);
}
int main()
{

    int n;
    cout << "Enter the number : ";
    cin >> n;
    cout << "Factorial of " << n << " is " << factorial(n);

    return 0;
}
```

TEST CASES:

Case 1;

Input = 5

```
Enter the number : 5
Factorial of 5 is 120
```

Case 2;

Input = 6

```
Enter the number : 6
Factorial of 6 is 720
```

Case 3;

Input = 0

```
Enter the number : 0
Factorial of 0 is 1
```

S.no: 4

Program Title: Program to check the number is odd or even

Source code

```
#include <iostream>
using namespace std;
bool isEven(int x)
{
    return x % 2 == 0;
}

int main()
{

    int x;
    cout << "Enter the number here : ";
    cin >> x;
    cout << x << " is " << (isEven(x) ? "even" : "odd");
    return 0;
}
```

TEST CASES:

Case 1;

Input = 10

```
Enter the number here : 10
10 is even
```

Case 2;

Input = 5

```
Enter the number here : 5
5 is odd
Enter the number here : 0
```

Case 3;

Input = 0

```
Enter the number here : 0
0 is even
```

S.no: 5

Program Title: Program using function

Source code

```
#include <iostream>
using namespace std;

int factorial(int n);

int main()
{

    int n;
    cout << "Enter the number : ";
    cin >> n;
    cout << "Factorial of " << n << " is " << factorial(n);

    return 0;
}

// Finding factorial using a function
int factorial(int n)
{
    if (n <= 1)
    {
        return 1;
    }
    else
    {
        return n * factorial(n - 1);
    }
}
```

TEST CASES:

Case 1;

Input = 6

```
Enter the number : 6
Factorial of 6 is 720
```

S.no: 6

Program Title: Program to find largest and smallest element in an array

Source code

```
#include <iostream>
#include "arrayprint.h"
using namespace std;

int min(int[], int);
int max(int[], int);
int main()
{
    // array input
    int arr[100], n;
    cout << "Enter number of elements: ";
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    cout << "Min : " << min(arr, n) << endl;
    cout << "Min : " << max(arr, n);
    return 0;
}
int min(int ar[], int size)
{
    int mn = ar[0];
    for (int i = 0; i < size; i++)
    {
        int c = ar[i];
        mn = (mn > c) ? c : mn;
    }
    return mn;
}
int max(int ar[], int size)
{
    int mx = ar[0];
    for (int i = 0; i < size; i++)
    {
        mx = (mx < ar[i]) ? ar[i] : mx;
    }
    return mx;
}
```

TEST CASES:

Case 1;

Input = [1,2,3]

```
Enter number of elements: 3
1
2
3
Min : 1
Min : 3
```

S.no: 7

Program Title: Program that finds the sum of numbers in an array.

Source code

```
#include <iostream>
using namespace std;
int sum(int ar[], int size)
{
    int sm = 0;
    for (int i = 0; i < size; i++)
    {
        sm += ar[i];
    }
    return sm;
}
int main()
{
    // array input
    int arr[100], n;
    cout << "Enter number of elements: ";
    cin >> n;

    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    cout << "Sum of the elements of the array : " << sum(arr, n);
    return 0;
}
```

TEST CASES:

Case 1;

Input = [1,2,3]

```
Enter number of elements: 3
1
2
3
Sum of the elements of the array : 6
```

S.no: 8

Program Title: Program that separates even and odd using an array

Source code

```
#include <iostream>
using namespace std;

void even_fliter(int arr[], int size)
{
    cout << "Even numbers are : ";
    for (int i = 0; i < size; i++)
    {
        if (arr[i] % 2 == 0)
        {
            cout << arr[i] << ",";
        }
    }
    cout << "\n-----------------\n";
}
void odd_fliter(int arr[], int size)
{
    cout << "Odd numbers are : ";
    for (int i = 0; i < size; i++)
    {
        if (arr[i] % 2)
        {
            cout << arr[i] << ",";
        }
    }
    cout << "\n-----------------\n";
}
int main()
{
    int arr[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

    even_fliter(arr, 10);
    odd_fliter(arr, 10);
    return 0;
}
```

Output

array = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

```
{ g++ array-even-odd.cpp -o array-ev
Even numbers are : 0,2,4,6,8,
-----------------
Odd numbers are : 1,3,5,7,9,
-----------------
```

S.no: 9

Program Title: Program that adds 2D arrays.

Source code

```
#include <iostream>
using namespace std;
void print(int ar[][3])
{
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            cout << ar[i][j] << "|";
        }
        cout << endl;
    }
}
void sum(int ar1[][3], int ar2[][3])
{
    int result[3][3];
    int range = 9;
    for (int i = 0; i < 9; i++)
    {
        result[i / 3][i % 3] = ar1[i / 3][i % 3] + ar2[i / 3][i % 3];
    }
    print(result);
}

int main()
{

    int ar1[3][3] = {{1, 2, 3}, {2, 3, 4}, {4, 5, 6}};
    int ar2[3][3] = {{3, 2, 1}, {4, 3, 2}, {6, 5, 4}};
    cout << "Array1" << endl;
    print(ar1);
    cout << "Array2" << endl;
    print(ar2);
    cout << "Sum " << endl;
    sum(ar1, ar2);
    return 0;
}
```

Output

```
Array1
1|2|3|
2|3|4|
4|5|6|
Array2
3|2|1|
4|3|2|
6|5|4|
Sum
4|4|4|
6|6|6|
10|10|10|
```

S.no: 10

Program Title: Program of Structure

Source code

```
#include <iostream>
using namespace std;

struct Person
{
    string name;
    int age;
    void show_details()
    {
        cout << name << "'s age is " << age;
    }
};
int main()
{
    struct Person p;
    p.name = "John";
    p.age = 10;
    p.show_details();
    return 0;
}
```

Output

```
John's age is 10
```

S.no: 11

Program Title: Program to print table of the given number by user.

Source code

```
#include <iostream>
using namespace std;

void table_print(int number)
{
    for (int i = 1; i <= 10; i++)
    {
        cout << number << " x " << i << " = " << number * i << endl;
    }
}

int main()
{
    int i;
    cout << "Enter the number here: ";
    cin >> i;
    table_print(i);
    return 0;
}
```

Test Cases

Case 1;

input =1

```
Enter the number here: 1
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
1 x 6 = 6
1 x 7 = 7
1 x 8 = 8
1 x 9 = 9
1 x 10 = 10
```

Case 2;

input =16

```
Enter the number here: 16
16 x 1 = 16
16 x 2 = 32
16 x 3 = 48
16 x 4 = 64
16 x 5 = 80
16 x 6 = 96
16 x 7 = 112
16 x 8 = 128
16 x 9 = 144
16 x 10 = 160
```

S.no: 12

Program Title: Program array passed by reference.

Source code

```cpp
#include <iostream>
using namespace std;

int double_elements(int array[], int size)
{
    for (int i = 0; i < size; i++)
    {
        array[i] = array[i] * 2;
    }
}
int main()
{
    int arr[3] = {1, 2, 3};
    cout << "Before the Call\n";
    for (int e : arr)
    {
        cout << e << ",";
    }
    double_elements(arr, 3);

    cout << "\nAfter the Call\n";
    // the array is modified here too
    for (int e : arr)
    {
        cout << e << ",";
    }
    return 0;
}
```

OUTPUT

```
Before the Call
1,2,3,
After the Call
2,4,6,
```

S.no: 13

Program Title: Program that demonstrates Structures in C++

```cpp
// Structure
#include <iostream>
using namespace std;
struct Person
{
    char name[20];
    int age;
};

void show_details(Person p)
{
    cout << p.name << "'s age is " << p.age << endl;
}

int main()
{
    Person people[3];
    for (int i = 0; i < 3; i++)
    {
        Person p;
        cout << "Enter name: ";
        cin >> p.name;
        cout << "Enter age: ";
        cin >> p.age;
        people[i] = p;
    }

    for (Person p : people)
    {
        show_details(p);
    }

    return 0;
}
```

OUTPUT

```
Enter name: Danny
Enter age: 22
Enter name: Raju
Enter age: 18
Enter name: Ron
Enter age: 12
Danny's age is 22
Raju's age is 18
Ron's age is 12
```

S.no: 14

Program Title: Class and object

```
// Class and object
#include <iostream>
using namespace std;
class Human
{
public:
    int age;
    string name;
    void introduce()
    {
        cout << "Age is " << age << endl;
        cout << "Name is " << name << endl;
    }
};
int main()
{
    Human h1;
    h1.age = 20;
    h1.name = "John";
    h1.introduce();
    return 0;
}
```

OUTPUT

```
Age is 20
Name is John
```

S.no: 15

Program Title: Program that uses Access Specifiers

```
// Access specifier

#include <iostream>
using namespace std;

class A
{
private:
    int pri;

protected:
    int pro;

public:
    int pub;
    void show()
    {
        cout << "public : " << pub << endl;
        cout << "protected : " << pro << endl;
        cout << "private : " << pri << endl;
    }
};
class B : public A
{
public:
    void modify()
    {
        pub = 1;
        pro = 3;
        // pri = 2; //not accessible
    }
};

int main()
{
    A obj;
    obj.pub = 1;
    // obj.pri = 2; //not accessible
    // obj.pro = 3; //not accessible
```

```
    obj.show();

    B obj1;
    obj1.modify();
    obj1.show();
    return 0;
}
```

OUTPUT

```
public : 1
protected : 0
private : 17569456
public : 1
protected : 3
private : 6422356
```

S.no: 16

Program Title: Program to demonstrate single level inheritance

```cpp
// Single Level Inheritance
#include <iostream>
using namespace std;

class Animal
{
public:
    string name;
    int age;
    void details()
    {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};
class Cat : public Animal
{
public:
    void meow()
    {
        cout << "Meow" << endl;
    }
};

int main()
{
    Cat c1;
    c1.name = "Tom";
    c1.age = 2;
    c1.details();
    c1.meow();

    return 0;
}
```

OUTPUT

```
Name: Tom
Age: 2
Meow
```

Danishwer

0026MCA25

S.no: 17
Program Title: Program that demonstrates multiple inheritance

```
// Multiple Inheritance
#include <iostream>
using namespace std;

class Human
{
public:
    int age;
    Human(int age) : age(age) {}
    virtual void introduce()
    {
        cout << "Age is " << age << endl;
    }
};
class Employee
{
public:
    string name;
    float salary;
    string company;

    Employee(string name, float salary, string company)
        : name(name), salary(salary), company(company) {}
    void introduce()
    {
        cout << "Name is " << name << endl;
        cout << "Salary is " << salary << endl;
        cout << "Company is " << company << endl;
    }
};
class Engineer : public Human, public Employee
{
public:
    int experience;
    Engineer(string name, int age, float salary, string company, int
experience) : Human(age), Employee(name, salary, company)
    {
        this->experience = experience;
    }
    void introduce() override
    {
        cout << "Name is " << name << endl;
```

```cpp
        cout << "Age is " << age << endl;
        cout << "Salary is " << salary << endl;
        cout << "Company is " << company << endl;
        cout << "Experience is " << experience << endl;
    }
};

int main()
{
    Engineer s1("Danny", 20, 100000, "Google", 5);
    s1.introduce();
    return 0;
}
```

OUTPUT

```
Name is Danny
Age is 20
Salary is 100000
Company is Google
Experience is 5
```

S.no: 18

Program Title: Program that demonstrates multilevel inheritance

```cpp
// Multilevel inheritance
#include <iostream>
using namespace std;
class Animal
{
public:
    void eat()
    {
        cout << "Eating" << endl;
    }
};
class LandAnimal : public Animal
{
public:
    void walk()
    {
        cout << "Walking" << endl;
    }
};
class Dog : public LandAnimal
{
public:
    void bark()
    {
        cout << "Barking" << endl;
    }
};
int main()
{
    Dog d;
    d.eat();
    d.walk();
    d.bark();
    return 0;
}
```

OUTPUT

```
Eating
Walking
Barking
```

Danishwer

0026MCA25

S.no: 19

Program Title: Program that demonstrates Hierarchical inheritance

```cpp
// Hierarchical Inheritance
#include <iostream>
using namespace std;
class Shape
{
public:
    int width, height;
    Shape(int w, int h)
    {
        width = w;
        height = h;
    }
};
class Rectangle : public Shape
{
public:
    Rectangle(int w, int h) : Shape(w, h) {}
    int area()
    {
        return width * height;
    }
};
class Triangle : public Shape
{
public:
    Triangle(int w, int h) : Shape(w, h) {}
    int area()
    {
        return (width * height) / 2;
    }
};

int main()
{
    Rectangle r(10, 20);
    Triangle t(10, 20);
    cout << r.area() << endl;
    cout << t.area() << endl;
    return 0;
}
```

OUTPUT

```
200
100
```

S.no: 20

Program Title: Program that demonstrates constructor

```cpp
// Constructor
#include <iostream>
using namespace std;

class Person
{
    string name;
    int age;

public:
    Person(string n, int a)
    {
        cout << "Constructor called" << endl;
        name = n;
        age = a;
    }
    void introduce()
    {
        cout << "Name is " << name << endl;
        cout << "Age is " << age << endl;
    }
};

int main()
{

    Person p1("John", 20); // constructor called
    p1.introduce();
    return 0;
}
```

OUTPUT

```
Constructor called
Name is John
Age is 20
```

Danishwer

0026MCA25

S.no: 21

Program Title: Program that demonstrates Polymorphism in C++

```cpp
// Polymorphism
#include <iostream>
using namespace std;

class Adder
{
public:
    static int add(int a, int b)
    {
        return a + b;
    }
    static int add(int a, int b, int c)
    {
        return a + b + c;
    }
    static double add(double a, double b)
    {
        return a + b;
    }
};

int main()
{
    cout << Adder::add(1, 2) << endl;
    cout << Adder::add(1, 2, 3) << endl;
    cout << Adder::add(1.0, 2.0) << endl;
    return 0;
}
```

OUTPUT

```
3
6
3
```

S.no: 22

Program Title: Program that demonstrates Function overloading in C++

```
// Function overloading
#include <iostream>
using namespace std;

int add(int a, int b)
{
    return a + b;
}
int add(int a, int b, int c)
{
    return a + b + c;
}
double add(double a, double b)
{
    return a + b;
}
int main()
{
    cout << add(10, 20) << endl;
    cout << add(10, 20, 30) << endl;
    cout << add(10.0, 20.0) << endl;
    return 0;
}
```

OUTPUT

```
30
60
30
```

Danishwer

0026MCA25

S.no: 23

Program Title: Program that demonstrates Class hierarchies in C++

```
// Class hierarchies:
// Hybrid inheritance and Dimond problem
#include <iostream>
using namespace std;
class Animal
{
public:
    void eat()
    {
        cout << "Eating" << endl;
    }
};

// using virtual inheritance to avoid diamond problem
class LandAnimal : virtual public Animal
{
public:
    void walk()
    {
        cout << "Walking" << endl;
    }
};
class Mammal : virtual public Animal
{
public:
    void feed()
    {
        cout << "Feeding" << endl;
    }
};

class Cat : public Mammal, public LandAnimal
{
public:
    void meow()
    {
        cout << "Meow" << endl;
    }
};

int main()
```

```
{
    Cat c1;
    c1.eat();
    c1.walk();
    c1.feed();
    c1.meow();
    return 0;
}
```

OUTPUT

```
Eating
Walking
Feeding
Meow
```

SQL

Danishwer

0026MCA25

DDL Commands

```sql
-- Create a database
CREATE DATABASE EMPLOYEE;
```

Logs:

```
Executing: CREATE DATABASE EMPLOYEE
Result: 1 rows affected in 13ms
```

```sql
-- Use a database
USE EMPLOYEE;
```

Logs:

```
Executing: USE EMPLOYEE
Result: 0 rows affected in 5ms
```

```sql
-- Drop a database
DROP DATABASE Employee;
```

Logs:

```
Executing: DROP DATABASE Employee
Result: 0 rows affected in 32ms
```

```sql
-- Create a table in a database
CREATE TABLE students (

   roll_no INT AUTO_INCREMENT PRIMARY KEY,

   name VARCHAR(50),

   department VARCHAR(50),

   city VARCHAR(50),

   marks FLOAT

);
```

```sql
-- Display all tables in a database
SHOW TABLES;
```

Tables_in_company_db varchar
students

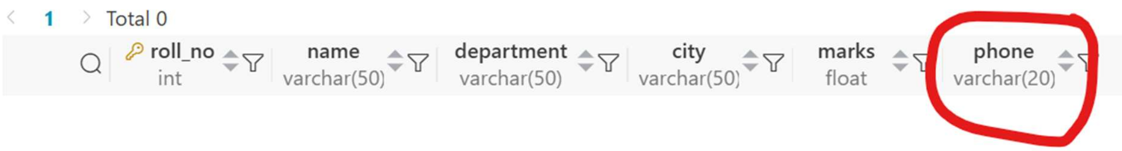
```sql
-- Describe a table : Used to display the structure of a table
DESCRIBE students;
```

Field varchar	Type blob
roll_no	int
name	varchar(50)
department	varchar(50)
city	varchar(50)
marks	float

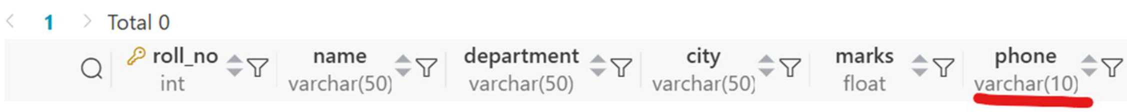
```sql
--Drop: Used to delete a table
DROP TABLE students;
```

```sql
-- Turncate: Used to empty a table
TRUNCATE TABLE students;
```

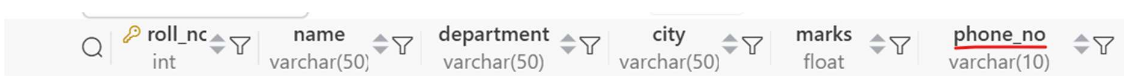
```sql
-- Alter

--Adding a new column
ALTER TABLE students ADD COLUMN phone VARCHAR(20);
```



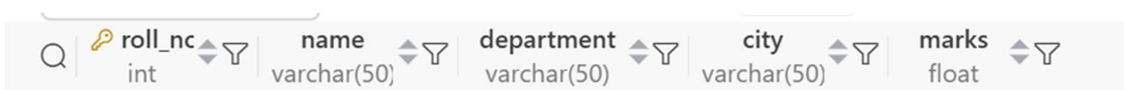

```sql
-- Modify a column
ALTER TABLE students MODIFY COLUMN phone VARCHAR(10);
```




```sql
-- rename a column
ALTER TABLE students RENAME COLUMN phone TO phone_no;
```




```sql
-- Drop a column
ALTER TABLE students DROP COLUMN phone_no;
```



Danishwer

0026MCA25

Table to run the following Queries on:

```sql
-- Select Data
SELECT * FROM students;
```

roll_no int	name varchar(50)	department varchar(50)	city varchar(50)	marks float
1	Aditi	CS	Delhi	85
2	Manav	IT	Mumbai	76
3	Riya	CS	Delhi	92
4	Kabir	Math	Pune	88
5	Sneha	IT	Mumbai	64
6	Arjun	Math	Delhi	91

1. Write a query to count students based on city

```sql
SELECT city, count(*) as student_count FROM students GROUP BY city;
```

city varchar(50)	student_count
Delhi	3
Mumbai	2
Pune	1

Danishwer

0026MCA25

2. Write a query to find the max marks for each department

```sql
SELECT department, max(marks) as max_marks

FROM students

GROUP BY

  department;
```

department varchar(50)	max_marks
CS	92
IT	76
Math	91

Danishwer

0026MCA25

3. Write a query to find the departments having average marks greater than 80.

```
Select department, avg(marks) as avg_marks

FROM students

GROUP BY

   department

HAVING

   AVG(marks) > 80;
```

department varchar(50)	avg_marks
CS	88.5
Math	89.5

4. Write a query to delete all students of the IT department.

```
DELETE FROM students WHERE department = 'IT';
```

```
Result: 3 rows retrieved in 3ms
Executing: DELETE FROM students WHERE department = 'IT'
```

5. Write a query to rename a column

```
Alter TABLE students RENAME COLUMN name TO s_name;
```

roll_nc	s_name	department	city	marks
int	varchar(50)	varchar(50)	varchar(50)	float

6. Write a query to show average marks for each department

```
Select department, avg(marks) as avg_marks

FROM students

GROUP BY

    department;
```

department varchar(50)	avg_marks
CS	88.5
IT	70
Math	89.5

Danishwer

0026MCA25

Create a table named **STUDENTS** with the following structure:

Column Name	Data Type	Description
roll_no	INT	Primary key, not null
name	VARCHAR(50)	Student name
department	VARCHAR(20)	Department name
city	VARCHAR(20)	City of the student

7. Create the STUDENTS table with the above structure.

```
CREATE Table Students (
    roll_no int PRIMARY KEY NOT NULL,
    name VARCHAR(50),
    department VARCHAR(20),
    city VARCHAR(20)
);
```

Cost: 21ms < 1 > Total 0

roll_no int	name varchar(50)	department varchar(20)	city varchar(20)

8. Add a new column email VARCHAR(50) to the table.

```sql
ALTER TABLE students ADD COLUMN email VARCHAR(50);
```

roll_no int	name varchar(50)	department varchar(20)	city varchar(20)	email varchar(50)

9. Modify the size of the `city` column to `VARCHAR(30)`.

```sql
ALTER Table students MODIFY COLUMN city VARCHAR(30);
```

Total 0

roll_no int	name varchar(50)	department varchar(20)	city varchar(30)	email varchar(50)

10. Rename the column name to student_name.

```sql
ALTER Table students MODIFY COLUMN city VARCHAR(30);
```

Total 0

roll_no int	student_name varchar(50)	department varchar(20)	city varchar(30)	email varchar(30)

Danishwer

0026MCA25

11.Add a UNIQUE constraint on `email`.

```
ALTER TABLE students MODIFY COLUMN email VARCHAR(30) UNIQUE;
```

CONSTRAINT_NAME varchar(64)	* CONSTRAINT_TYPE varchar(11)
email	UNIQUE

12.Drop the column `city`.

```
ALTER Table students DROP COLUMN city;
```

roll_no int	student_name varchar(50)	department varchar(20)	email varchar(30)

13.Rename the table STUDENTS to STUDENT_RECORDS.

```
ALTER TABLE students RENAME students_record;
```

Danishwer

0026MCA25

14. Delete all rows using (Explain differences):

1. **TRUNCATE**
2. **DELETE**

```
TRUNCATE TABLE students_record;
```

```
DELETE FROM students_record WHERE 1 = 1;
```

```
Executing: DELETE FROM students_record
Result: 0 rows affected in 11ms
Executing: TRUNCATE TABLE students_record
Result: 0 rows affected in 38ms
```

The main difference between Truncate and Delete is that:

1. Delete removes and log rows one by one with a condition and Truncate drops every tuple at once.
2. Delete can be rolled back but truncate can't be.
3. Delete is the part of DML and Truncate is part of DDL.

15. Drop the entire table.

```
DROP TABLE students_record;
```

```
Executing: DROP TABLE students_record
Result: 0 rows affected in 19ms
```